第2章 线性表

本章的目的是？ → 解决线性表的计算机实现问题

↓

2.1 线性表的定义和特点（逻辑结构）

2.2 线性表的顺序存储实现（顺序表）

2.3 线性表的链式存储实现（链表）

本章重点难点

重点:

(1)线性表顺序存储;

(2)线性表的链式存储;

(3)双向链表;

(4)循环链表。

⇔

特点、基本操作和有关算法

难点: 线性表链式存储实现，包括单链表、双向链表、循环链表的基本操作和有关算法。

第2章 线性表

2.1 线性表的类型定义

2.2 线性表的顺序表示和实现

2.3 线性表的链式表示和实现

2.4 一元多项式的表示

2.1 线性表的类型定义

□线性表的定义----什么是线性表？

姓名	年龄	性别	成绩	
张三	20	男	86	→ a_1
李四	19	男	85	→ a_2
王五	20	男	84	→ a_3
……	……	……	……	
麻n	20	男	59	→ a_n

2.1 线性表的类型定义

□线性表的定义----什么是线性表？

线性表是n 个类型相同数据元素的有限序列，通常记作（$a_1, a_2, a_3, \ldots, a_n$）。

□线性结构的基本特征为:----线性表有哪些特征？

(1)集合中必存在唯一的一个“第一元素”；

(2)集合中必存在唯一的一个“最后元素”；

(3)除最后元素之外，均有 唯一的后继；

(4)除第一元素之外，均有 唯一的前驱。

2.1 线性表的类型定义

□线性结构的抽象数据类型描述

ADT List {

数据对象： $D=\{ a_i \mid a_i \in ElemSet, i=1,2,\ldots,n, \ n\geq 0 \}$

{称n 为线性表的表长;

称 n=0 时的线性表为空表。}

数据关系： $R1=\{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, \ i=2,\ldots,n \}$

基本操作： 见下页

}ADT List

2.1 线性表的类型定义

□ 基本操作

InitList(&L)　　//构造一个空的线性表L。

DestroyList(&L)　//销毁线性表 L

ListEmpty(L)　　//判断栈L是否空

ListLength(L)　　//求L的长度

PriorElem(L, cur_e, &pre_e)　//求前驱的值

NextElem(L, cur_e, &next_e) //求后继的值

2.1 线性表的类型定义

基本操作

GetElem(L, i, &e)　　//取i位置的值

LocateElem(L, e, compare( ))　//在线性表中查找e

ListTraverse(L, visit( ))　//遍历线性表

ClearList(&L)　　//清空线性表

ListInsert(&L, i, e)　　//在i位置插入e

ListDelete(&L, i, &e)　　//删除i位置的元素。

算法的描述

输入 → 算法 → 输出

算法描述的一般格式

```
返回值   算法对应的函数名(形参列表)
{        //临时变量的定义
         //实现由输入参数到输出参数的操作
         …
}
```

函数体

- 返回值：通常为bool类型，表示算法是否成功执行。
- 形参列表：由输入型参数和输出型参数构成。

输入型参数 ↕ 算法输入　　输出型参数 ↕ 算法输出

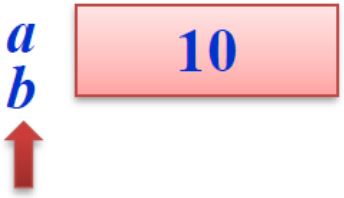
如何描述输出型参数？

C++语言中提供了一种引用运算符“&”用于描述输出型参数。

引用示例

```
int a=10;
int &b=a;
```

引用



两个变量共享内存空间

a、*b*同步发生改变

示例：设计一个交换两个整数的算法。

编写一个函数swap1(*x*，*y*)：

```
void swap1(int x,  int y)
{    int tmp;
     tmp=x;  x=y;  y=tmp;
}
```

交换形参*x*和*y*的值

当执行语句swap1(*a*,*b*)时，*a*和*b*实参值不会发生了交换。

改正方法1：采用指针的方式来回传形参的值，需将上述函数改为：

```
void swap2(int *x，int *y)
{    int tmp;
     tmp=*x;          //将x的值放在tmp中
     *x=*y;           //将x所指的值改为*y
     *y=tmp;        //将y所指的值改为tmp
}
```

交换形参*x*和*y*所指向的值

上述函数的调用改为swap2(&*a*,&*b*)

改正方法2：采用引用型形参 ⇨ 将输出型形参改为引用类型。

```
void  swap(int &x,  int &y)
//形参前的“&”符号不是指针运算符
{    int tmp=x;
     x=y; y=tmp;
}
```

交换形参*x*和*y*的值

当执行语句swap(*a*,*b*)时，形、实参的匹配相当于：

```
int  &x=a;    //a为x的引用
int  &y=b;    //b为y的引用
```

这样，*a*与*x*共享存储空间、*b*与*y*共享存储空间，因此执行函数后*a*和*b*的值发生了交换 ⇨ 简单。

□基本操作的应用

用定义过的基本操作实现例2-1，例2-2

例 2-1

假设:有两个集合 A 和 B 分别用两个线性表 LA 和 LB 表示，即：线性表中的数据元素即为集合中的成员。

现要求一个新的集合A＝A∪B。

□例2-1操作步骤：

(1) 从线性表LB中依次察看每个数据元素;

GetElem(LB, i,&e)

(2) 依值在线性表LA中进行查访;

LocateElem(LA, e, equal( ))

(3) 若不存在，则插入LA中。

ListInsert(LA, n+1, e)

2.1 线性表的类型定义

□例2-1算法

```
void union(List &La, List Lb) {
    La_len = ListLength(La);        // 求线性表的长度
    Lb_len = ListLength(Lb);

   for (i = 1;  i <= Lb_len;  i++) {

   ……................................

    }

  }
```

2.1 线性表的类型定义

□例2-1算法

```
GetElem(Lb, i, &e);
            // 取Lb中第i个数据元素赋给e
if (!LocateElem(La, e, equal( )) )
  ListInsert(La, ++La_len, e);
            // La中不存在和 e 相同的数据元素，
              则插入之
```

□例2-1算法时间复杂性

O(ListLength(La)*ListLength(Lb))

2.1 线性表的类型定义

□基本操作的应用

用定义过的基本操作实现例2-1，例2-2

例 2-2

将两个“数据元素按值递增有序排列”的线性表La和Lb归并到线性表Lc，且Lc具有同样的性质。

设 $La=(a_1,\ldots a_i,\ldots.a_n)$

$Lb=(b_1,\ldots b_j,\ldots.b_m)$

$Lc=(c_1,\ldots c_k,\ldots c_{m+n})$

2.1 线性表的类型定义

□例2-2算法

```
InitList(Lc);  // 构造空的线性表 Lc
i = j = 1;    k = 0;
La_len = ListLength(La);
Lb_len = ListLength(Lb);

while ((i <= La_len) && (j <= Lb_len))
    {  // La 和 Lb 均不空 }
while (i<=La_len)  // 若 La 不空
    {……}
while (j<=Lb_len)  // 若 Lb 不空
    {……}
```

La: x_1 x_2 x_3 x_4 x_5

i ↑

Lb: y_1 y_2 y_3 y_4 y_5

j ↑

若x_1<= y_1

Lc: x_1

k ↑

见下页

□例2-2算法

```
GetElem(La, i, &ai);
GetElem(Lb, j, &bj);
if (ai <= bj) {          // 将 ai 插入到 Lc 中
  ListInsert(Lc, ++k, ai);
  ++i; }
else {                   // 将 bj 插入到 Lc 中
  ListInsert(Lc, ++k, bj);  ++j; }
```

2.1 线性表的类型定义

□例2-2算法

```
while (i <= La_len) {      // 当La不空时
    GetElem(La, i++, ai);
    ListInsert(Lc, ++k, ai);
}                          // 插入 La 表中剩余元素

while (j <= Lb_len) {      // 当Lb不空时
    GetElem(Lb, j++, bj);
    ListInsert(Lc, ++k, bj);
}                          // 插入 Lb 表中剩余元素
```

- 例2-2算法时间复杂性

O(ListLength(La)+ListLength(Lb))

第2章 线性表

2.1 线性表的类型定义

2.2 线性表的顺序表示和实现

2.3 线性表的链式表示和实现

2.4 一元多项式的表示

2.2 线性表的顺序表示和实现

存储线性表，就是要保存至少两类信息：

(1) 线性表中的数据元素；

(2) 线性表中数据元素的顺序关系。

2.2 线性表的顺序表示和实现

□线性表的顺序表示的定义

以数据元素x 的存储位置和它的下一个数据元素 y 的存储位置之间的某种关系表示逻辑关系<x, y>。

线性表的顺序表示最简单的一种顺序映象方法是：令x的存储位置和y 的存储位置相邻。

2.2 线性表的顺序表示和实现

□线性表的顺序表示图示

用一组地址连续的存储单元依次存放线性表中的数据元素

	a_1	a_2	…	a_{i-1}	a_i	…	a_n	

线性表的起始地址
称作线性表的基地址

2.2 线性表的顺序表示和实现

□数据元素地址之间的关系

以“存储位置相邻”表示有序对<a_{i-1}，a_i>时，a_{i-1}和a_i的地址关系如下：

$$LOC(a_i) = LOC(a_{i-1}) + C$$

C为一个数据元素所占存储量

所有数据元素的存储位置均取决于第一个数据元素的存储位置。

$$LOC(a_i) = LOC(a_1) + (i-1)\times C$$

↑基地址

2.1 线性表的类型定义

□顺序表的实现 → 第1步！

例1 实现下表的顺序存储

姓名	年龄	性别	成绩
张三	20	男	86
李四	19	男	85
王五	20	男	84
……	……	……	……
麻n	20	男	59

1、数据元素的数据类型Elemtype实现

方式1

```
struct  data {
char  name[8];
int    age;
char  sex[2];
float  score;
};
typedef  struct data  Elemtype;
```

2.1 线性表的类型定义

- 顺序表示的实现

第2步！

2、顺序表数据类型的实现

姓名	年龄	性别	成绩
张三	20	男	86
李四	19	男	85
王五	20	男	84
……	……	……	……
麻n	20	男	59

方式1：静态数组实现

```
typedef  struct{
    Elemtype   data[30];
    int  length;
    int  listsize;
}SqList;
```

SqList现在是一个顺序表数据类型；

2.1 线性表的类型定义

□顺序表示的实现 第3步！

姓名	年龄	性别	成绩
张三	20	男	86
李四	19	男	85
王五	20	男	84
……	……	……	……
麻n	20	男	59

3、顺序表的实现

SqList L；

第i个人的信息存储在：

L.data[i-1].name； //姓名

L.data[i-1].age； //年龄

L.data[i-1].sex； //性别

L.data[i-1].score； //分数

表信息：

L.length； // 线性表长度

L.listsize； // 线性表规模

2.2 线性表的顺序表示和实现

□顺序表示的实现 顺序表数据类型的动态数组实现

```
#define  LIST_INIT_SIZE     80
                // 线性表存储空间的初始分配量
#define  LISTINCREMENT    10
                // 线性表存储空间的分配增量

typedef  struct {

    ElemType *elem;     // 存储空间基址
    int   length;       // 当前长度
    int   listsize;     // 当前分配的存储容量
                        // (以sizeof(ElemType)为单位)
} SqList;               // 俗称 顺序表
```

2.2 线性表的顺序表示和实现

□基本操作的实现---- 构造一个空的线性表

```
Status InitList_Sq( SqList    &L ) {

L.elem = (ElemType*) malloc (LIST_INIT_SIZE*sizeof (ElemType));
if (!L.elem)
    exit(OVERFLOW);

  L.length = 0;
  L.listsize = LIST_INIT_SIZE;
  return OK;


  } // InitList_Sq
```

注意！应用时：
先声明线性表变量
Sqlist L;
然后调用函数
InitList_Sq(L);

2.2 线性表的顺序表示和实现

□基本操作的实现---- 销毁线性表

```
Status DestroyList( SqList    &L ) {
    free(L.elem);
    return ok;
} // InitList_Sq
```

2.2 线性表的顺序表示和实现

□基本操作的实现---- 判断线性表是否为空

```
Status ListEmpty( SqList    L ) {
        if(L.length==0)
           return TRUE;
        Else
           return FALSE;

} // InitList_Sq
```

2.2 线性表的顺序表示和实现

□基本操作的实现---- 求L的长度

```
Status ListLength( SqList    L ) {

        return  L.length;

} // InitList_Sq
```

顺序表基本操作的实现

LocateElem(L, e, compare( )) //在线性表中查找e

ListTraverse(L, visit( )) //遍历线性表

ClearList(&L) //清空线性表

ListInsert(&L, i, e) //在i位置插入e

ListDelete(&L, i, &e) //删除i位置的元素。

□基本操作的实现

线性表操作

ListInsert(&L, i, e)的实现：

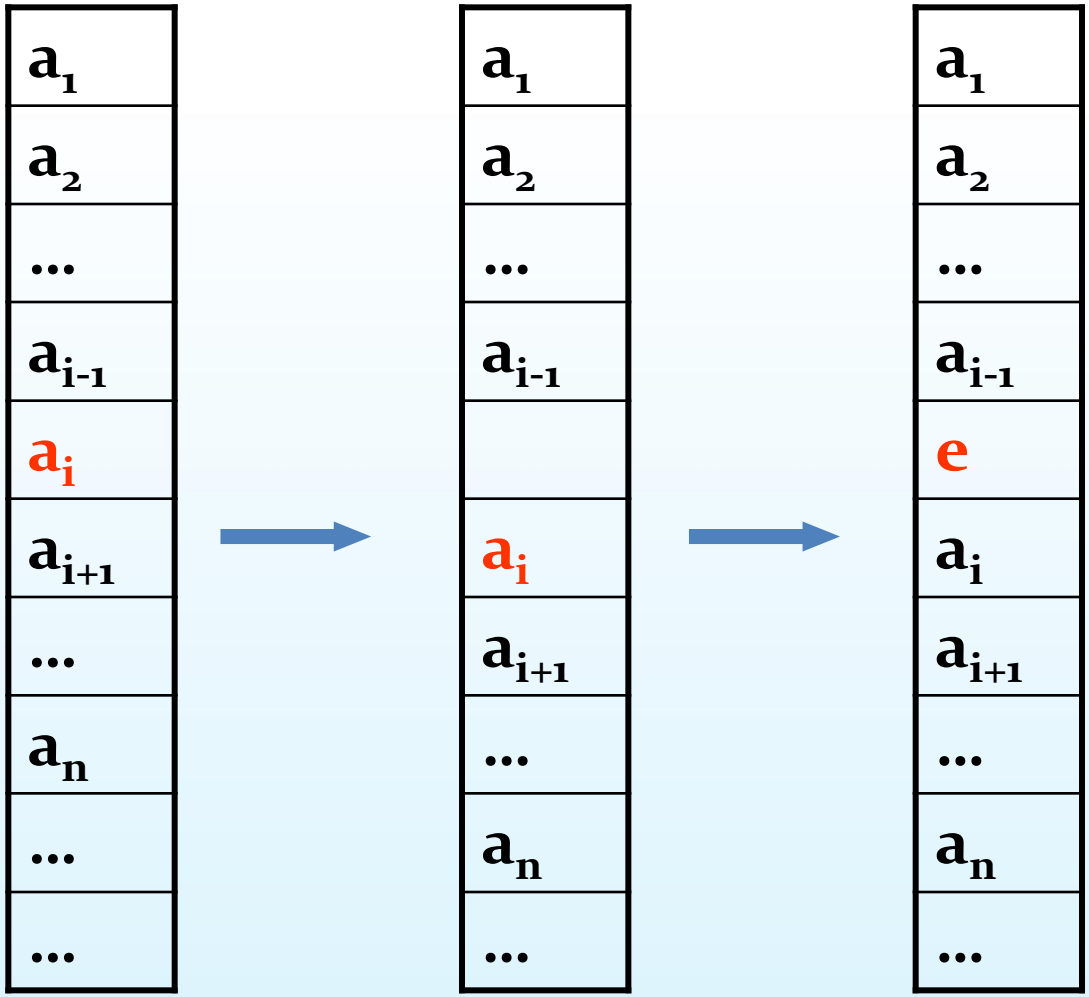
首先分析:

插入元素时，

线性表的逻辑结构发生什么变化？

2.2 线性表的顺序表示和实现

□插入操作过程示意图



□插入操作算法

```
Status ListInsert_Sq(SqList &L, int i, ElemType e) {
        // 在顺序表L的第 i 个元素之前插入新的元素e

    if (i < 1 || i > L.length+1) return ERROR;
                                    // 插入位置不合法

} // ListInsert_Sq
```

2.2 线性表的顺序表示和实现

□插入操作算法

```
Status ListInsert_Sq(SqList &L, int i, ElemType e) {
    if (L.length >= L.listsize) {
                                    // 当前存储空间已满，增加分配
        newbase = (ElemType *)realloc(L.elem,
            (L.listsize+LISTINCREMENT)*sizeof (ElemType));
        if (!newbase)
            exit(OVERFLOW);
        L.elem = newbase; // 新基址
        L.listsize += LISTINCREMENT;
    .........
    }
}
```

2.2 线性表的顺序表示和实现

□插入操作算法--数组实现!

```
Status ListInsert_Sq(SqList &L, int i, ElemType e) {
   // 在顺序表L的第 i (1≤i≤n)个元素之前插入新的元素e
   ……

  for (j = L.length-1; j >= i-1;  --j)
      L.elem[j+1]=L.elem[j];// 插入位置及之后的元素右移
  L.elem[i-1]=e; // 插入e
  ++L.length;          // 表长增1
  return OK;

} // ListInsert_Sq
```

2.2 线性表的顺序表示和实现

□插入操作算法—指针实现!

```
Status ListInsert_Sq(SqList &L, int i, ElemType e) {
        // 在顺序表L的第 i 个元素之前插入新的元素e
   ……
q = &(L.elem[i-1]);     // q 指示插入位置
for (p = &(L.elem[L.length-1]); p >= q;  --p)
    *(p+1) = *p;            // 插入位置及之后的元素右移
    *q = e;                 // 插入e
     ++L.length;            // 表长增1
return OK;
} // ListInsert_Sq
```

单选题 1分

设置

在一个长度为n的顺序存储的线性表中，向第i个元素（1 ≤i ≤n+1）位置插入一个新元素时，需要从后向前依次后移（ ）个元素。

A n-i

B n-i+1

C n-i-1

D i

提交

顺序表插入算法复杂度分析

- 对于本算法来说，元素移动的次数不仅与表长有关，而且与插入位置i有关：
- 算法最好时间复杂度为O(1)
  - 当$i=n+1$时，移动次数为0
- 算法最坏时间复杂度为O(n)
  - 当$i=1$时，移动次数为n，达到最大值

□插入算法时间复杂性分析

设在第i个元素之前插入的概率为p_i，则在长度为n的线性表中插入一个元素时所需平均移动元素次数为：

$$E_{IS}(n)=\sum_{i=1}^{n+1}p_i(n-i+1)$$

若假定在任何一个位置上插入元素的概率相同，则平均移动次数为：

$$E_{IS}(n)=\sum_{i=1}^{n+1}\frac{(n-i+1)}{n+1}=\frac{n}{2}$$

O(n)

□基本操作的实现

线性表删除操作ListDelete(&L, i, &e)的实现：

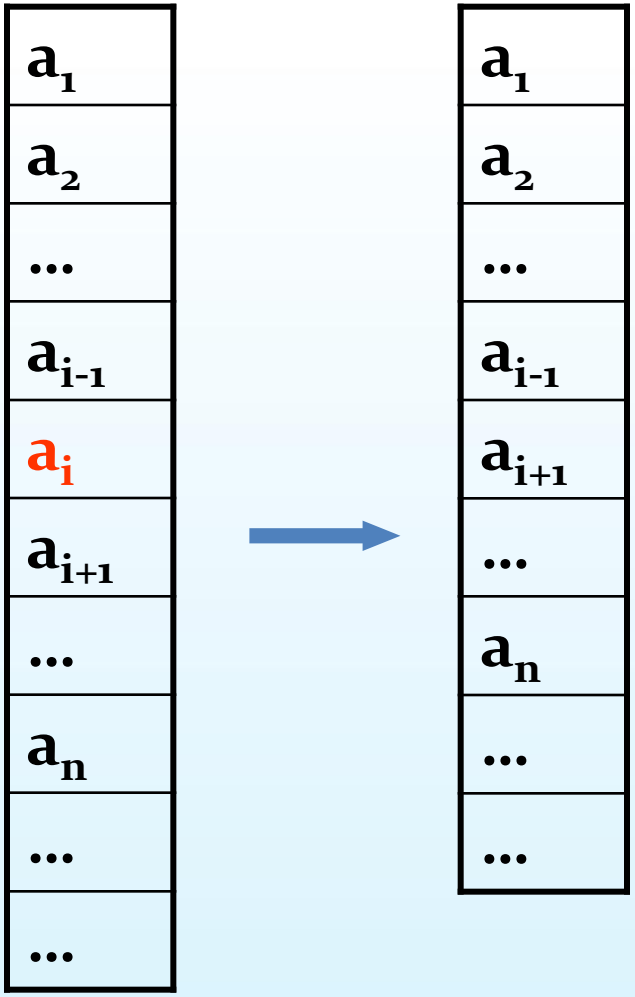
首先分析：

删除元素时，

线性表的逻辑结构发生什么变化？

2.2 线性表的顺序表示和实现

□删除操作过程示意图



2.2 线性表的顺序表示和实现

□删除算法----判断删除位置

```
Status ListDelete_Sq
                    (SqList &L, int i, ElemType &e) {


    if ((i < 1) || (i > L.length))
    return ERROR;


} // ListDelete_Sq
```

2.2 线性表的顺序表示和实现

□删除算法----数组实现!

```
Status ListDelete_Sq
                    (SqList &L, int i, ElemType &e) {
……................
  e = L.elem[i-1];   //记录删除的元素

  for (j=i-1; j <L.length-1; j++)
     L.elem[j]=L.elem[j+1];
  --L.length;                    // 表长减1
  return OK;

} // ListDelete_Sq
```

2.2 线性表的顺序表示和实现

□删除算法----指针实现!

```
Status ListDelete_Sq
                    (SqList &L, int i, ElemType &e) {
……................
  p = &(L.elem[i-1]);          // p 为被删除元素的位置
  e = *p;                      // 被删除元素的值赋给 e
  q = L.elem+L.length-1;       // 表尾元素的位置
  for (++p; p <= q; ++p)
      *(p-1) = *p;
  --L.length;                  // 表长减1
  return OK;
} // ListDelete_Sq
```

单选题 1分

设置

在一个长度为n的顺序存储的线性表中，删除第i个元素（1 ≤i ≤n+1）时，需要从前向后依次前移（ ）个元素。

A n-i

B n-i+1

C n-i-1

D i

提交

□删除操作时间复杂性分析

- ◆对于本算法来说，元素移动的次数也与表长n和删除元素的位置i有关：
- ◆当$i=n$时，移动次数为0；
  - ➢最好情况：时间复杂度为O(1)
- ◆当$i=1$时，移动次数为$n-1$。
  - ➢最坏情况：时间复杂度为O(n)

□删除操作时间复杂性分析

设删除第i个元素的概率为p_i，则在长度为n的线性表中删除一个元素时所需平均移动次数为：

$$E_{IS}(n)=\sum_{i=1}^{n}p_i(n-i)$$

若假定在任何一个位置上删除元素的概率相同，则平均移动次数为：

$$E_{IS}(n)=\sum_{i=1}^{n}\frac{(n-i)}{n}=\frac{n-1}{2}$$

O(n)

2.1 线性表的类型定义

□ 静态数组和动态数组的说明

静态数组

```
typedef  struct{
   Elemtype   data[30];
   int  length;
   int  listsize;
}SqList;
```

动态数组

```
typedef  struct{
   ElemType *elem;
   int  length;
   int  listsize;
}SqList;
```

Sqlist L；//声明线性表变量L

Sqlist L；

L.data是数组名

L.elem = (ElemType*) malloc (30*sizeof (ElemType));

L.elem是数组名

□顺序表的优缺点

(1) 优点

a.节省存储空间;

b.对线性表中第i个结点的存取操作易于实现;

c.容易查找一个结点的前驱和后继。

(2)缺点

a.插入和删除操作需要移动数据;

b.建立空表时，较难确定所需的存储空间。

第2章 线性表

2.1 线性表的类型定义

2.2 线性表的顺序表示和实现

2.3 线性表的链式表示和实现

2.4 一元多项式的表示

2.3 线性表的链式表示和实现

2.3.1 单链表

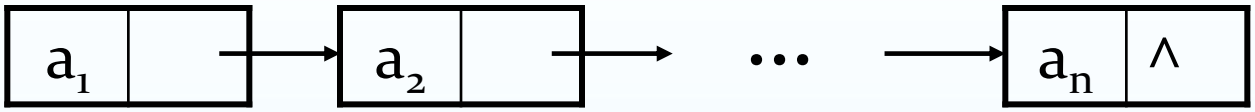
2.3.2 静态链表

2.3.3 循环链表

2.3.4 双向链表

2.3.1 单链表

□单链表结构示意图



□链表的定义

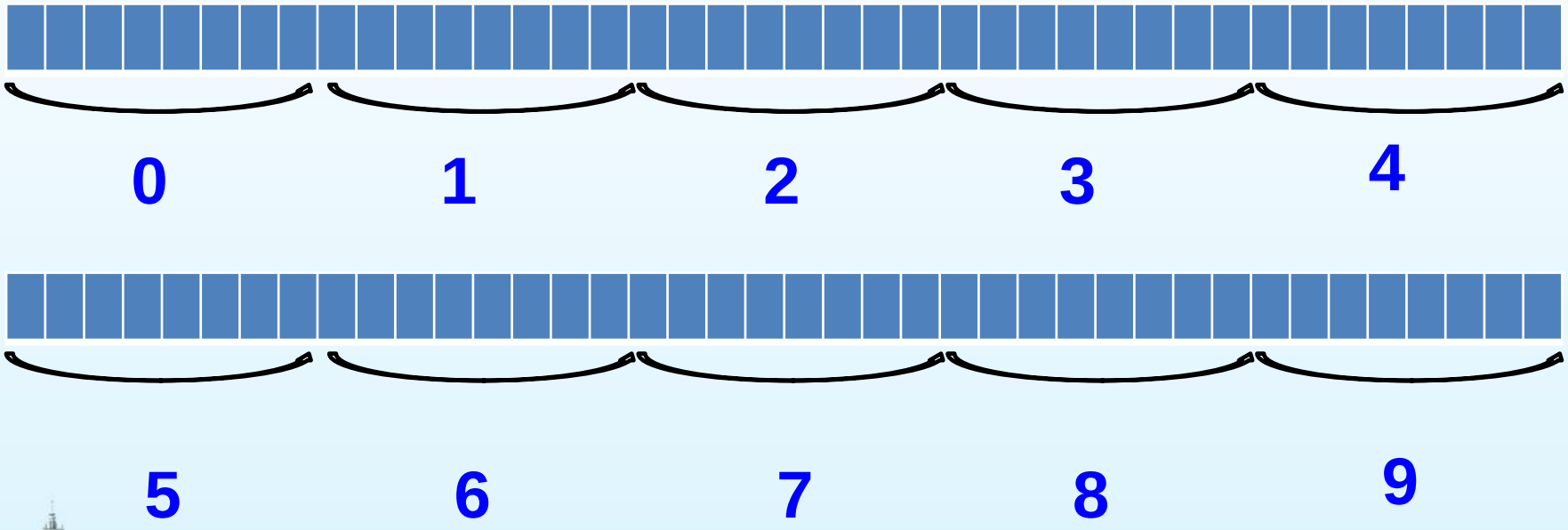
一个线性表由若干个结点组成，每个结点至少含有两个域：*数据域(信息域)和指针域(链域)*，由这样的结点存储的线性表称作链表。

2.3.1 单链表

□计算机内存地址补充说明

计算机内存像仓库一样，由“位”（bit）构成，每一位可以存一个“0”或“1”，每8位称作一个字节（byte），每一个字节有一个编号，这个编号称作是这个字节的地址。

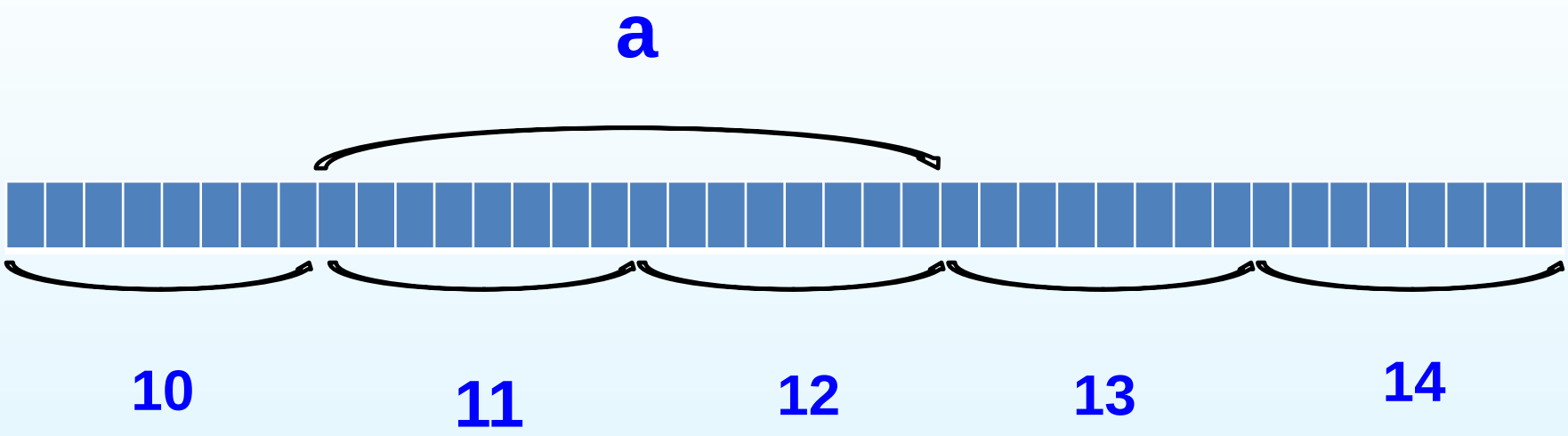
1K=1024 字节， 1M=1024K，1G=1024 M



2.3.1 单链表

□计算机内存地址补充说明

当运行int a; 时，以int类型占用16位为例，为变量a分配了2个字节的空间，例如下图：

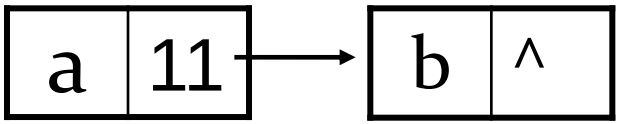


a的首地址是11，称作a的地址是11.

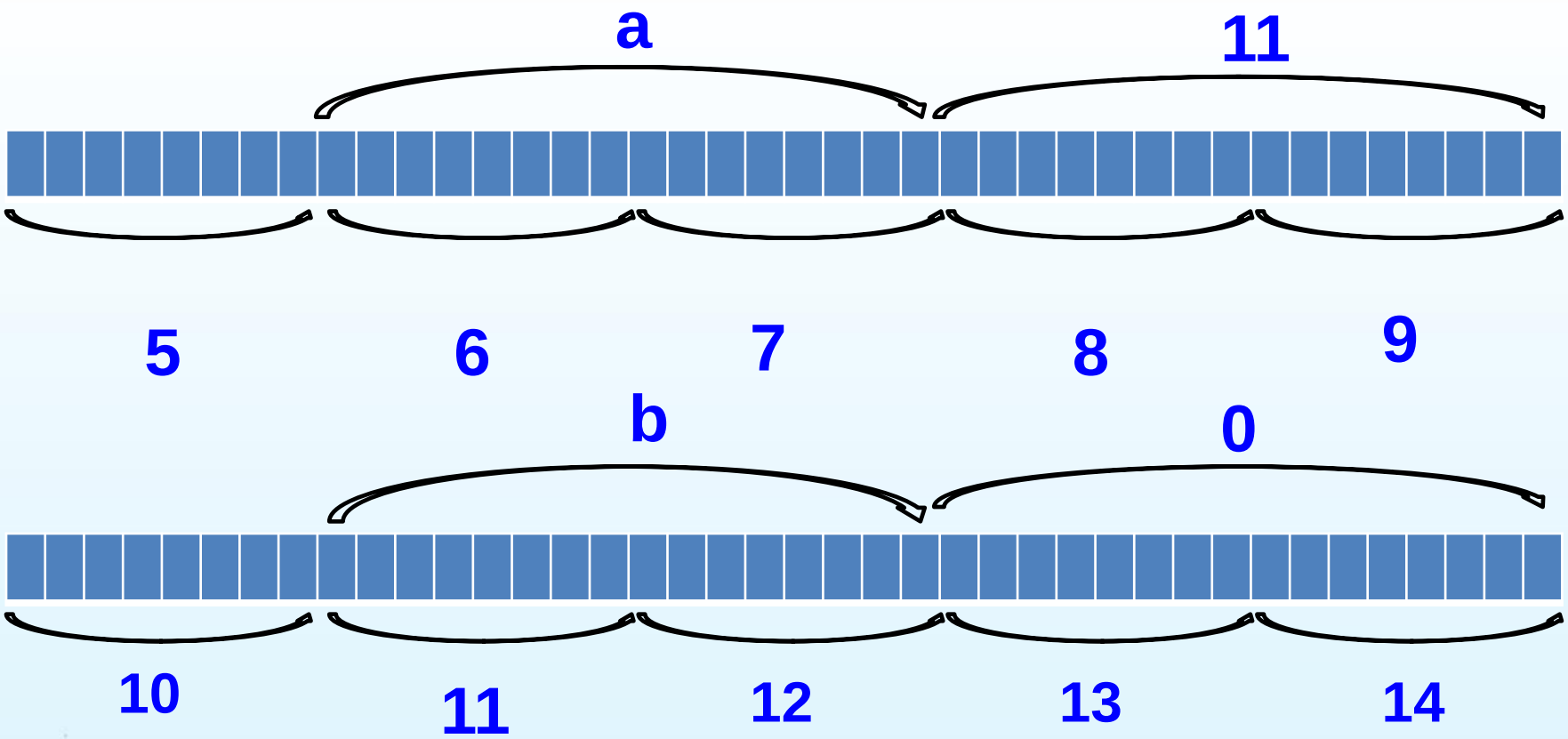
2.3.1 单链表

□计算机内存地址补充说明

例如有链表：



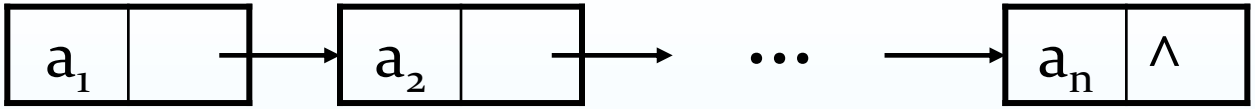
设地址占16位



通过这个例子，说明一下链表的缺点。

2.3.1 单链表

□单链表结构示意图



□结构特点

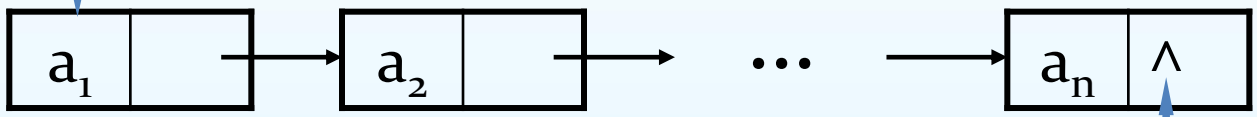
逻辑次序和物理次序不一定相同；

元素之间的逻辑关系用指针表示；需要额外空间存储元素之间的关系。

2.3.1 单链表

□头指针的概念

在链式存储结构中以第一个结点的存储地址作为线性表的基地址，通常称它为头指针。

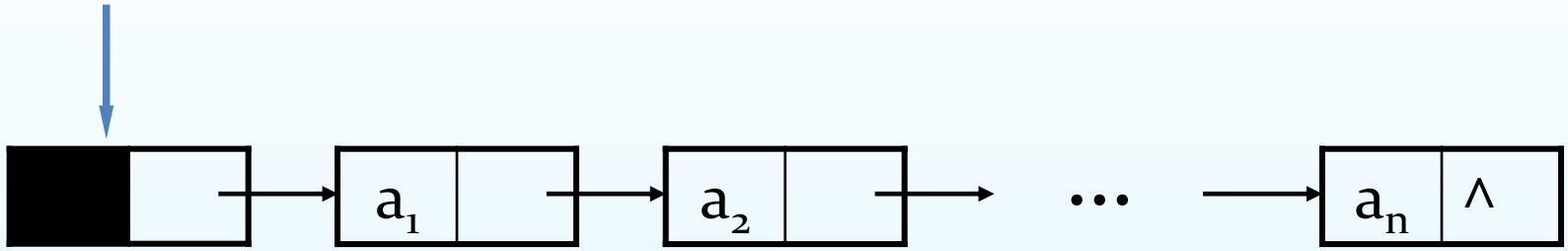


线性表的最后一个数据元素没有后继，因此最后一个结点中的“指针域"是一个特殊的值 "NULL"，通常称它为"空指针"。

2.3.1 单链表

□头结点的概念

在单链表上设置一个结点，它本身不存放数据，它的指针域指向第一个元素的地址。



□ 头结点的作用

使对第一个元素的操作与对其它元素的操作保持一致。

2.3.1 单链表

□单链表的C语言实现

```
Typedef struct  LNode {
  ElemType        data;   // 数据域
  struct LNode    *next;  // 指针域
} LNode, *LinkList;
```

数据类型LNode和指针数据类型LinkList 说明

LinkList和Lnode *是一样的

节点数据类型的实现

如果我们要实现逻辑结构

第一种方式：LNode L;

两个域的表示方法分别为：
数据域 L.data;
指针域 L.next;

第二种方式：
LNode *L等价于 LinkList L

L = (LNode*) malloc(sizeof (LNode));

两个域的表示方法分别为：
数据域 L->data;
指针域 L->next;

单链表基本操作的实现分析

InitList(&L) //构造一个空的带头节点的线性表L。

```
LinkList  L;
void InitList(LinkList &L){
L=(Lnode *) malloc (sizeof (Lnode));
L->next=NULL;
}
```

单链表基本操作的实现分析

InitList(&L) //构造一个空的不带头节点的线性表L。

```
LinkList  L;

void InitList(LinkList &L){

L=NULL;

}
```

单链表基本操作的实现分析

InitList(&L) //构造一个空的带头节点的线性表L。

```
LinkList  L;
void InitList(LinkList &L){
    L=(Lnode *) malloc (sizeof (Lnode));
    L->next=NULL;
}
```

ListEmpty(L)
//判断线性表L是否空

```
int ListEmpty( LinkList  L ){
    if (L->next==NULL)
        return TRUE;
    else
        return  FALSE;
}
```

单选题 1分

设置

带头结点的单链表H为空的判定条件是（ ）。

A H==NULL

B H->next==NULL

C H->next==H

D H!=NULL

提交

单链表基本操作的实现分析

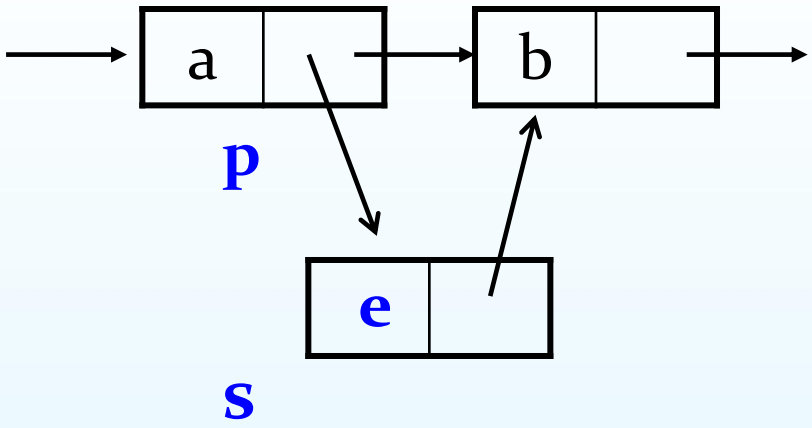
InitList(&L) //构造一个空的不带头节点的线性表L。

```
LinkList  L;
void InitList(LinkList &L){
L=NULL;
}
```

ListEmpty(L)
//判断线性表L是否空

```
int ListEmpty( LinkList  L ){
    if (L==NULL)
        return TRUE;
    else
        return  FALSE;
}
```

单链表插入节点示意图
—在a节点后插入一个节点，其值为e



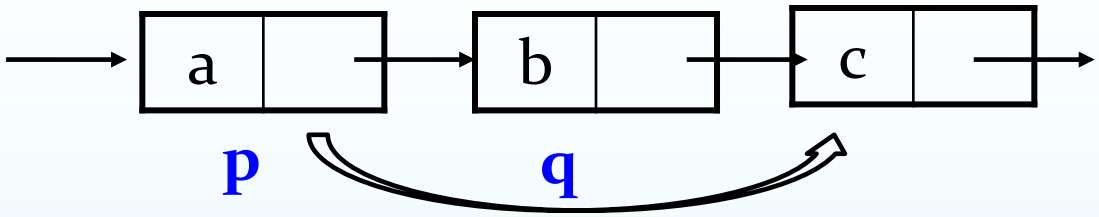
注意！必须知道a节点的地址，设为p

```
LinkList s=(Lnode *) malloc (sizeof (Lnode));
s->data=e;
s->next=p->next;
p->next=s;
```

问题：
最后两句颠倒一下次序是否可以？

```
p->next=s;
s->next=p->next;
```

单链表删除节点示意图
—在a节点后删除一个节点



注意！必须知道a的地址，设为p

```
q=p->next;
p->next=q->next;（或）p->next=p->next->next;
free(q);
```

单选题 1分

设置

以下关于线性表的链式存储结构的叙述中，正确的是（）。

A 存储密度大

B 逻辑上相邻的结点物理上不必相邻

C 可以通过计算直接确定第i个结点的存储地址

D 插入、删除运算操作不方便

提交

单选题 1分

设置

下面关于线性表的叙述中，错误的是哪一个？

A 线性表采用顺序存储，必须占用一片连续的存储单元。

B 线性表采用顺序存储，便于进行插入和删除操作。

C 线性表采用链接存储，不必占用一片连续的存储单元。

D 线性表采用链接存储，便于插入和删除操作。

提交

单链表基本操作的实现分析

DestroyList(&L) //销毁线性表 L

ListLength(L) //求L的长度

PriorElem(L, cur_e, &pre_e) //求前驱的值

NextElem(L, cur_e, &next_e) //求后继的值

单链表基本操作的实现分析

GetElem(L, i, &e)　　//取i位置的值

LocateElem(L, e, compare( ))　//在线性表中查找e

ListTraverse(L, visit( ))　//遍历线性表

ClearList(&L)　　//清空线性表

ListInsert(&L, i, e)　　//在i位置插入e

ListDelete(&L, i, &e)　　//删除i位置的元素。

2.3.1 单链表

□单链表操作的实现

GetElem_L(LinkList L, int i, ElemType &e)

//查找第i个结点，将第i个结点的值存入变量e

□查找过程

(1)从第一个结点p开始，j=1开始记数;

(2)当p->next不为空时，转向下一个继续记数;

(3)直到j=i或p->next为空。

2.3.1 单链表

□GetElem_L算法

```
Status GetElem_L(LinkList L, int i, ElemType &e) {

  p = L->next;   j = 1;  // p指向第一个结点，j为计数器

  while (p && j<i)
  { p = p->next;  ++j;  }

  if ( !p)
     return ERROR;      //  第 i 个元素不存在
  e = p->data;          //  取得第 i 个元素
  return OK;
}
```

2.3.1 单链表

□单链表操作的实现

```
Status ListInsert_L(LinkList L, int i, ElemType e)
// L 为带头结点的单链表的头指针，本算法
// 在链表中第i 个结点之前插入新的元素 e
```

□插入过程

(1)查找第i-1个结点p;

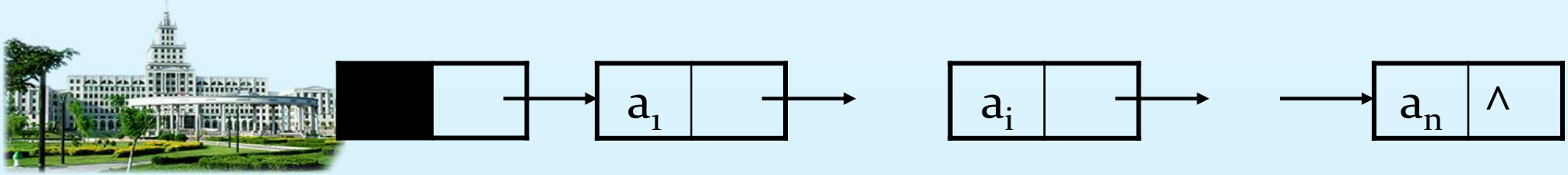
(2)生成结点s,存入e;

(3)s->next=p->next,p->next=s。

2.3.1 单链表

□ListInsert_L算法

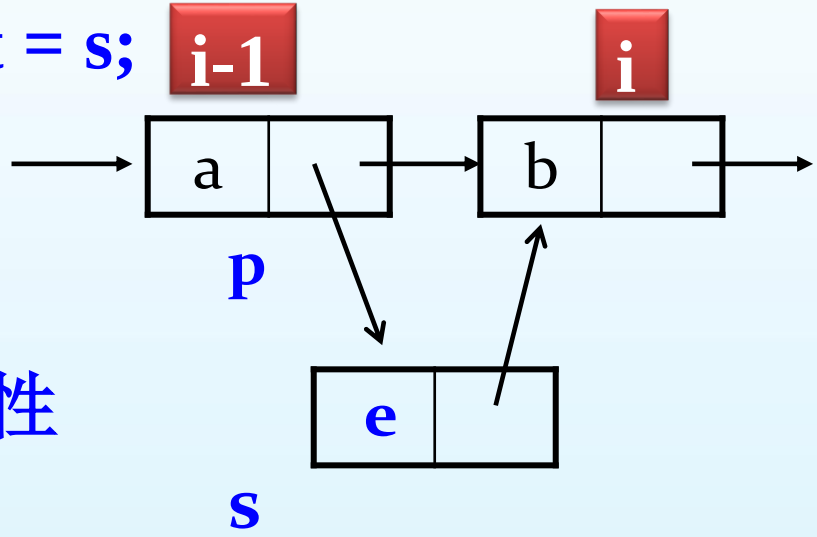
```
Status ListInsert_L(LinkList L, int i, ElemType e)
{    // L 为带头结点的单链表的头指针，本算法
  // 在链表中第i 个结点之前插入新的元素 e

 p = L;    j = 0;
 while (p && j < i-1)
     { p = p->next;  ++j; }   // 寻找第 i-1 个结点
 if (!p )
      return ERROR;          // i 大于表长或者小于1

  ……

} // LinstInsert_L
```



2.3.1 单链表

☐ListInsert_L算法

```
s = (LinkList) malloc ( sizeof (LNode));
                    // 生成新结点
s->data = e;
s->next = p->next;      p->next = s;
                    // 插入
return OK;
```



☐ ListInsert_L算法的时间杂性

O(ListLength(L))

2.3.1 单链表

□单链表操作的实现

ListDelete_L(LinkList L, int i, ElemType &e)
// L 为带头结点的单链表的头指针，本算法
// 删除链表中第i 个结点，值储入 e

□删除过程

(1) 查找第i-1个结点p；

(2) q=p->next,p->next=q->next

(3) free(q)

2.3.1 单链表

☐ListDelete_L算法

```
Status ListDelete_L(LinkList L, int i, ElemType &e) {

  p = L;    j = 0;
  while (p->next && j < i-1)
     {  p = p->next;   ++j; }
  if  (!(p->next) )
     return ERROR;      // 删除位置不合理

  q = p->next;   p->next = q->next; e = q->data;
  free(q);
  return OK;

} // ListDelete_L
```

2.3.1 单链表

□链表的优缺点

(1) 优点

a. 插入和删除不需要移动数据（时间复杂性低）；

b.不需预先分配空间。

(2)缺点

a.指针占用存储空间，增加了内存负担。

b.只能顺序查找（操作）。

2.3 线性表的链式表示和实现

2.3.1 单链表

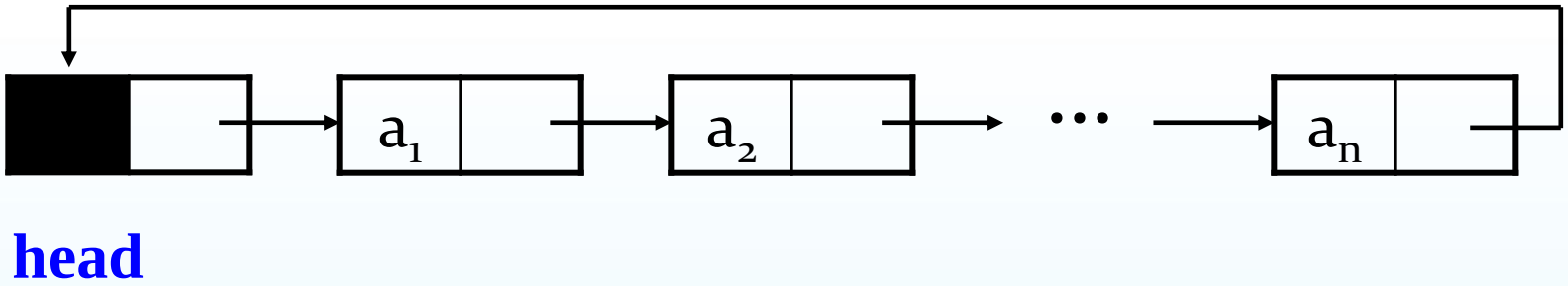
2.3.2 静态链表

2.3.3 循环链表

2.3.4 双向链表

2.3.3 循环链表

□循环链表示意图

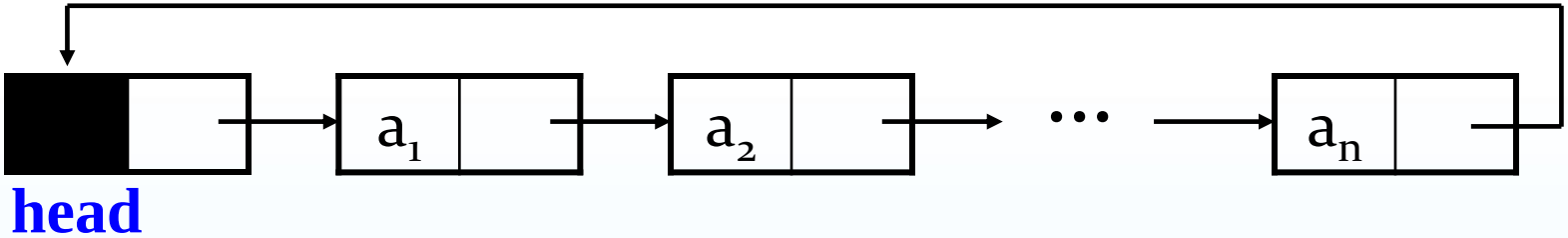


□循环链表的概念

单链表最后一个结点的指针域没有利用，如果使其指向头指针（头结点），则首尾构成一个循环，称作循环链表。

2.3.3 循环链表

□循环链表的优点

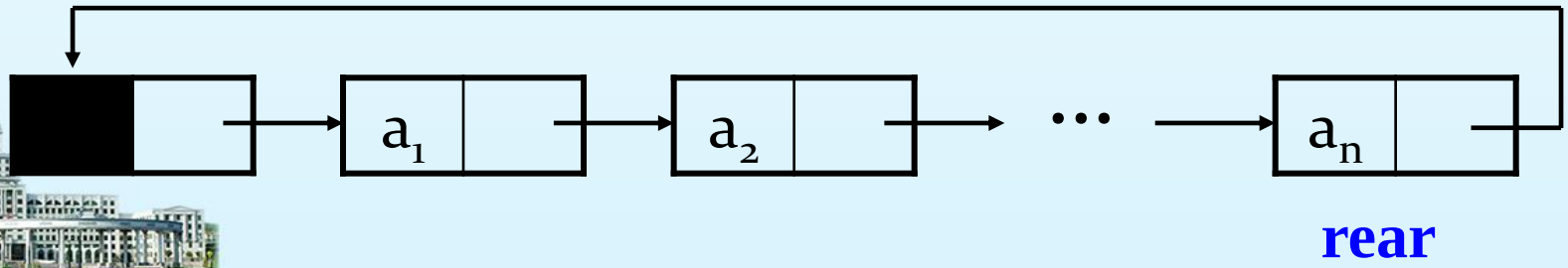


从表中任一结点出发均可找到表中其它结点。

讨论：在循环链表中多采用尾指针？为什么？

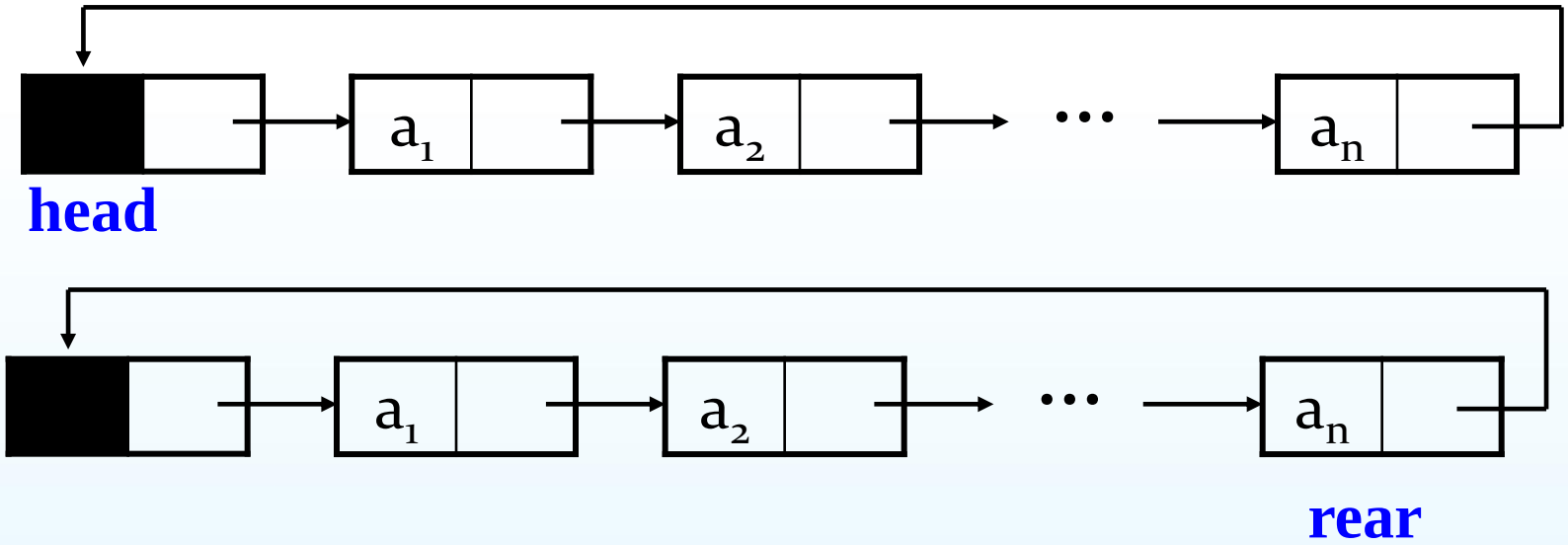
什么是尾指针？

指向表尾的指针称为是尾指针。



2.3.3 循环链表

□循环链表的优点



讨论：在循环链表中多采用尾指针？为什么？

结论：采用头指针时，查找第一个结点容易，查找尾结点不容易，如果采用尾指针，则查找第一个结点和最后一个结点都容易。

单选题 1分

设置

现有单链表L1中节点p和循环单链表L2中节点q，下面哪个判断p和q是尾节点的语句是正确的？

A if(p==NULL)；if(q==NULL) ;

B if(p->next==NULL)；if(q->next==NULL)

C if(p==NULL)；if(q->next=L2)

D if(p->next==NULL)；if(q->next==L2)

提交

2.3.3 循环链表

□单链表和循环链表操作比较

设两个链表La=(a_1,a_2,...,a_n)和Lb=(b_1,b_2,...b_m)，讨论如下问题：

(1) La、Lb都是带头结点的单链表，如何实现将Lb接在La之后？时间复杂度是多少？

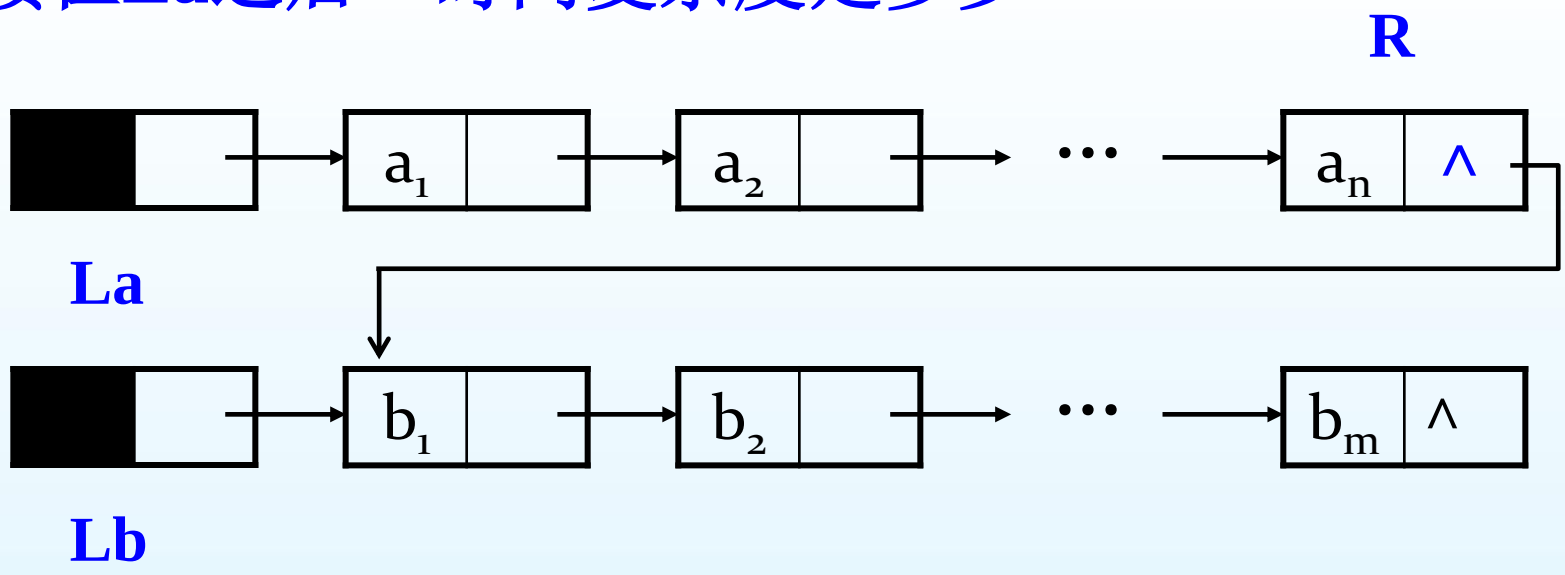
(2) La、Lb都是带头结点头指针的单循环链表，如何实现将Lb接在La之后还形成一个循环链表？时间复杂度是多少？

(3) La、Lb都是带头结点尾指针的单循环链表，如何实现将Lb接在La之后还形成一个循环链表？时间复杂度是多少？

2.3.3 循环链表

□单链表和循环链表操作比较

(1) La、Lb都是带头结点的单链表，如何实现将Lb接在La之后？时间复杂度是多少？

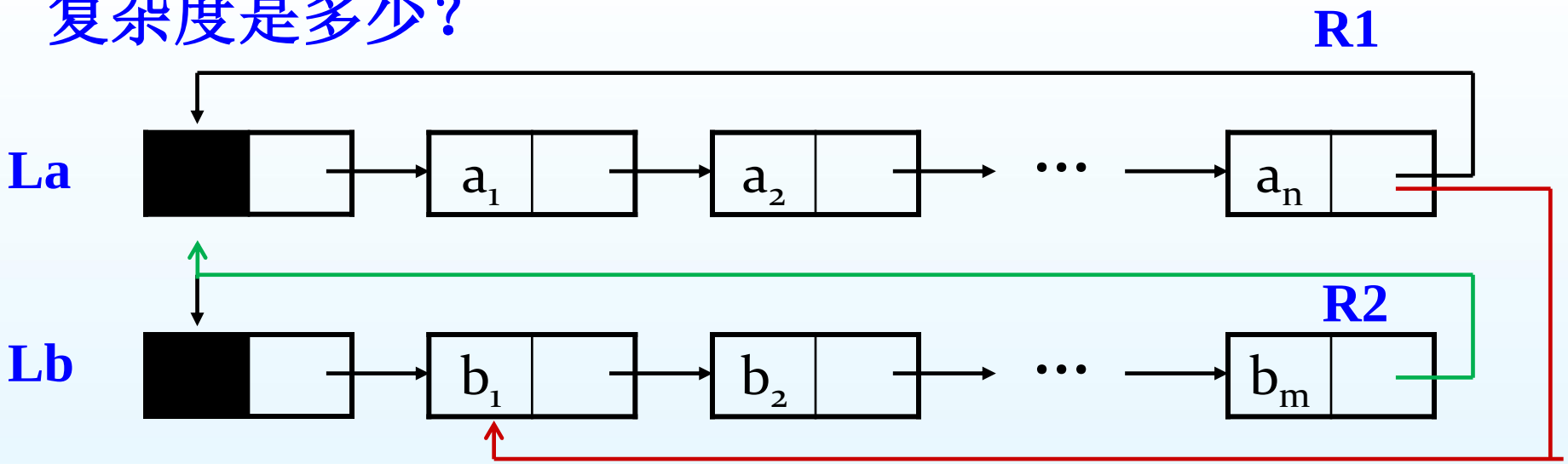



```
R=La;
While(R->next)
R=R->next;
R->next=Lb->next;
free(Lb)
```

2.3.3 循环链表

□单链表和循环链表操作比较

(2) La、Lb都是带头结点头指针的单循环链表，如何实现将Lb接在La之后还形成一个循环链表？时间复杂度是多少？




```
R1=La;
While(R1->next!=La)
R1=R1->next;
```

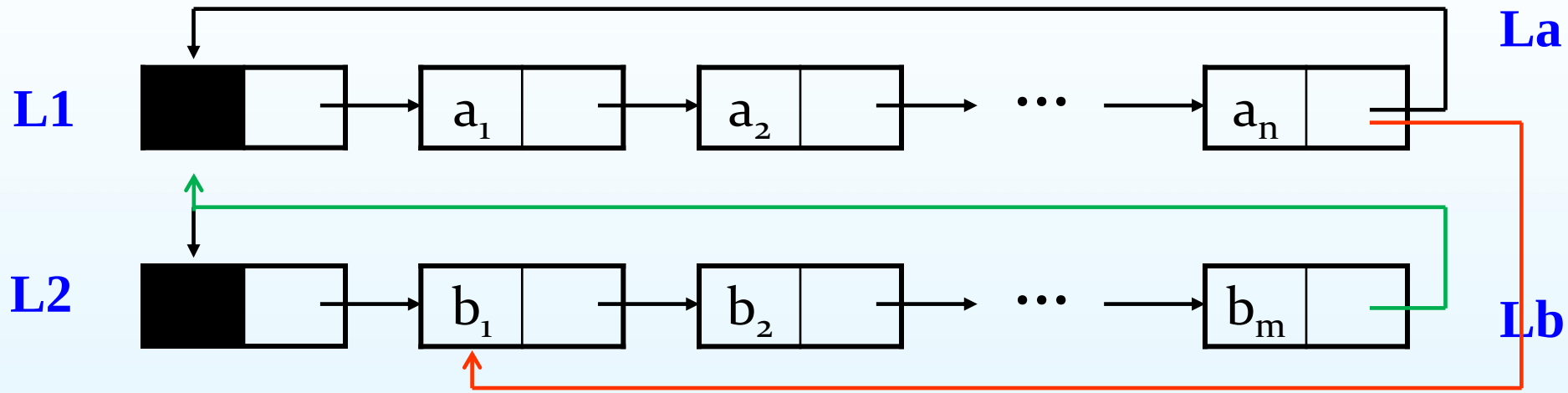
```
R2=Lb;
While(R2->next!=Lb)
R2=R2->next;
```

```
R1->next=Lb->next;
R2->next=La;
free(Lb)
```

2.3.3 循环链表

□单链表和循环链表操作比较

(3) La、Lb都是带头结点尾指针的单循环链表，如何实现将Lb接在La之后还形成一个循环链表？




```
L1=La->next;
L2=Lb->next;
La->next=L2->next;
```

```
Lb->next=L1;
free(L2)
```

2.3.3 循环链表

□单链表和循环链表操作比较

结论：

(1) La、Lb都是带头结点的单链表，Lb接在La之后，时间复杂度是 O(length(La))。

(2) La、Lb都是带头结点头指针的单循环链表，实现将Lb接在La之后还形成一个循环链表，时间复杂度是多少 O(length(La)+length(Lb))。

(3) La、Lb都是带头结点尾指针的单循环链表，实现将Lb接在La之后还形成一个循环链表，时间复杂度是 O(1)。

2.3 线性表的链式表示和实现

2.3.1 单链表

2.3.2 静态链表

2.3.3 循环链表

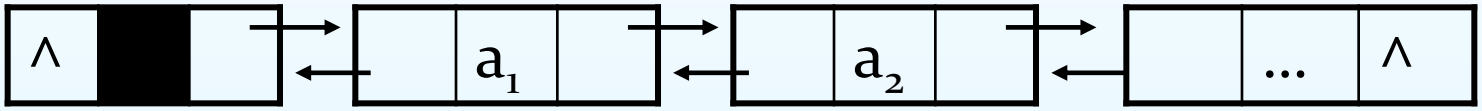
2.3.4 双向链表

2.3.4 双向链表

□ 双向链表的概念

一个链表的每一个结点含有两个指针域：一个指针指向其前驱结点，另一个指针指向其后继结点，这样的链表称为双向链表。

□双向链表结构示意图



head

优缺点：

优点是：可以双向查找线性表。

缺点是：进一步增加了内存负担，操作难度加大了。

2.3.4 双向链表

□ 双向链表的C语言实现

```
//类型定义
typedef struct DuLNode{
    ElemType  data;
    struct DuLNode *prior;
    struct DuLNode *next;
} DuLNode,*DuLinkList;
```

2.3.4 双向链表

- 双向链表中地址关系

在双向链表中：

p->prior指向p的前驱，p->next指向p的后继

以下都是p结点的地址：

p->next->prior 和 p->prior->next

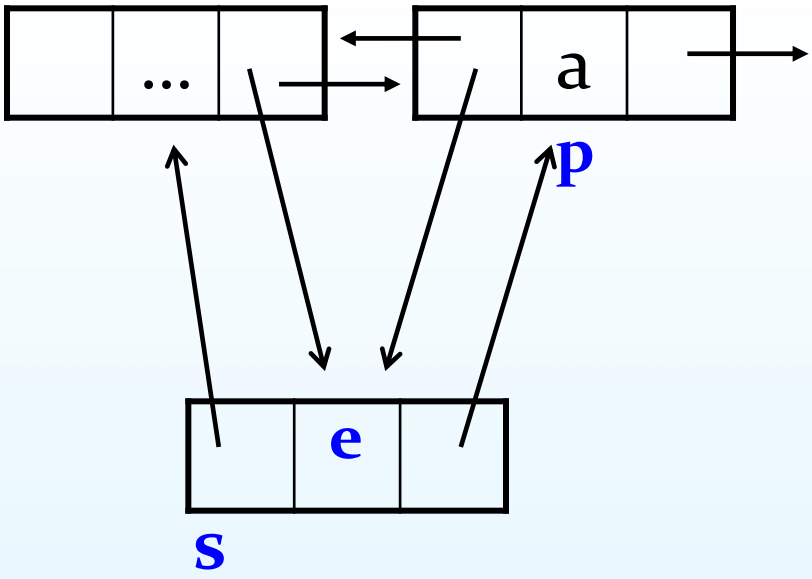
- 双向链表的操作特点：

“查询” 和单链表相同。

“插入” 和“删除”时需要同时修改两个方向上的指针。

- 双向链表插入节点示意图
  —在a节点前插入一个节点，其值为e

注意！必须知道a节点的地址，设为p

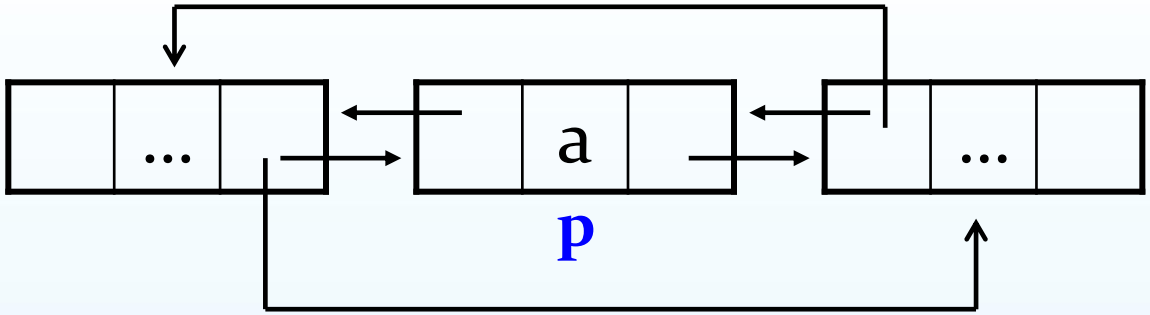



```
s->prior=p->prior;
s->next=p;
p->prior->next=s;
p->prior=s;
```

```
DuLinkList s=(DuLnode *) malloc (sizeof (DuLnode));
s->data=e;
```

- 双向链表删除节点示意图—删除a节点。

注意！必须知道a节点的地址，设为p




```
p->prior->next=p->next;
p->next->prior=p->prior;
free(p);
```

2.3.4 双向链表

□ 双向链表的插入算法

```
Status ListInsert_Dul(DuLinkList &L, int i ,ElemType e)
{    if(!(p=GetElemP_DuL(L,i)))
        return error;
     if(!(s=(DuLinkList)malloc(sizeof(DuLNode))))
        return error;
     s->data=e; s->prior=p->prior;
     s->next=p; p->prior->next=s;
     p->prior=s;
     return OK;
}
```

2.3.4 双向链表

□ 双向链表的删除算法

```
Status ListDelete_Dul(DuLinkList &L,int i ,ElemType &e)
{
    if(!(p=GetElemP_DuL(L,i)))
      return error;
    e=p->data;
    p->prior->next=p->next;
    p->next->prior=p->prior;
    free(p);
    return OK;
}
```

单选题 1分

设置

设一个链表最常用的操作是在末尾插入结点和删除尾结点，则选用(　　)最节省时间。

A 单链表

B 单循环链表

C 带尾指针的单循环链表

D 带头结点的双循环链表

提交

单选题 1分

设置

若某线性表最常用的操作是存取和在最后进行插入元素和删除运算，则利用（　）存储方式最节省时间。

A 顺序表

B 双链表

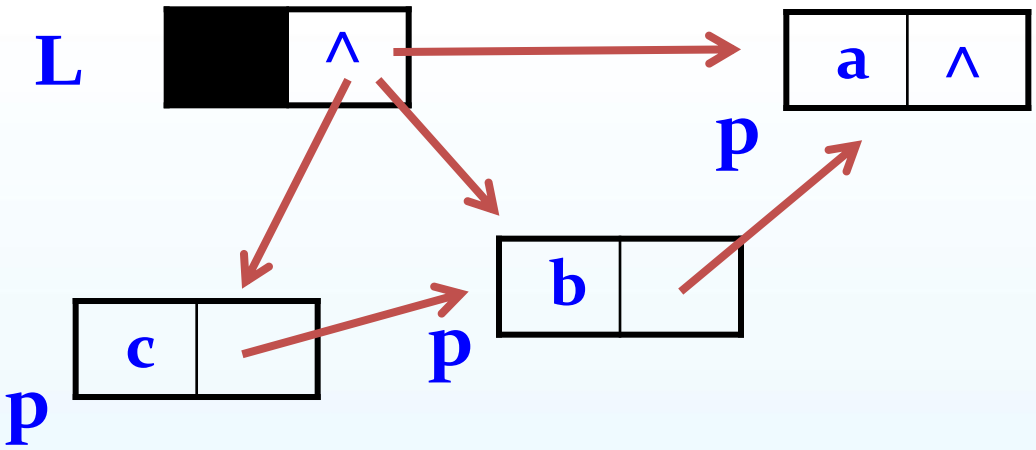
C 带头结点的双循环链表

D 单循环链表

提交

建立单链表 头插法：

- 例如：建立三个元素a,b,c的单链表

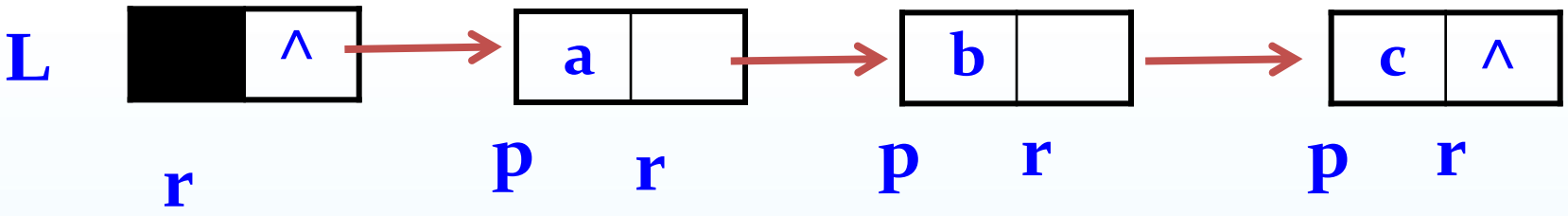


单链表的头插法建表有什么特点？

```
p=(LNode *) malloc (sizeof (LNode));
P->data=a;
p->next=L->next;
L->next=p;
```

建立单链表 ➡ 尾插法：

- 例如：建立三个元素a,b,c的单链表




```
r=L;
p=(LNode *) malloc (sizeof (LNode));
p->data=a;
r->next=p;
r=p;
r->next=NULL;
```

单链表的尾插法建表有什么特点？

第2章 线性表

2.1 线性表的类型定义

2.2 线性表的顺序表示和实现

2.3 线性表的链式表示和实现

2.4 一元多项式的表示

2.4 一元多项式的表示

□ 一元多项式的讨论

$$p_n(x) = p_0 + p_1 x + p_2 x^2 + \ldots + p_n x^n$$

讨论一元多项式用什么方式存储比较合适？

结论：一元多项式既可以用顺序表存储，也可以用链表存储，在用顺序表存储时，如果是稀疏多项式则浪费空间严重，用链式存储比较好。

2.4 一元多项式的表示

一元多项式的抽象数据类型

ADT Polynomial {

数据对象： $D=\{ a_i \mid a_i \in TermSet, i=1,2,...,m, m\geq 0$

TermSet 中的每个元素包含一个表示系数的实数和表示指数的整数 }

数据关系： $R1=\{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, \quad i=2,...,n$

且a_{i-1}中的指数值＜a_i中的指数值 }

基本操作： **见下页**

} ADT Polynomial

2.4 一元多项式的表示

◘一元多项式的基本操作

CreatPolyn (&P, m)　　//存储多项式

DestroyPolyn (&P)　　//销毁多项式

PrintPolyn (&P)　　//打印输出多项式

PolynLength(P)　　//求多项式的项数

AddPolyn (&Pa, &Pb)　　//两个多项式相加

SubtractPolyn (&Pa, &Pb)//两个多项式相减

……

2.4 一元多项式的表示

□一元多项式类型的C语言实现

(1) 顺序存储表示的类型

```
typedef struct
   { float  coef;   /*系数部分*/
     int    expn;   /*指数部分*/
   } ElemType ;

typedef  struct
{   ElemType  a[MAX_SIZE] ;
    int   length ;
}Sqlist  ;
```

(2) 链式存储表示的类型

```
typedef struct ploy
   {   float coef ;    /*系数部分*/
       int   expn ;    /*指数部分*/
       struct ploy  *next ;
   } Ploy ;
```

一元多项式相加：

- 指数不同： 链表的合并。
- 指数相同： 系数相加，和为0，去掉结点，和不为0，修改结点的系数域。

算法：

对两个多项式链表进行相加，生成一个新的相加后的结果多项式链表，原来两个多项式链表依然存在。

算法描述

```
ploy *add_ploy(ploy *La, ploy *Lb)
  /* 将以La，Lb为头指针表示的一元多项式相加，生成一个新的结果多项式 */
  { ploy *Lc,*pc,*pa,*pb,*p; float x;
    Lc=pc=(ploy *)malloc(sizeof(ploy));
    pa=La->next; pb=Lb->next;
    while (pa!=NULL && pb!=NULL)
      { if (pa->expn<pb->expn)
          { p=(ploy *)malloc(sizeof(ploy));
            p->coef=pa->coef; p->expn=pa->expn;
            p->next=NULL;
```

```
        /* 生成一个新的结果结点并赋值 */
      pc->next=p ; pc=p ; pa=pa->next ;
    }
     /* 生成的结点插入到结果链表的最后，pa指向下一个结点 */
if  (pa->expn>pb->expn)
  {  p=(ploy *)malloc(sizeof(ploy)) ;
      p->coef=pb->coef ; p->expn=pb->expn ;
      p->next=NULL ;
        /* 生成一个新的结果结点并赋值 */
      pc->next=p ; pc=p ; pb=pb->next ;
    }   /* 生成的结点插入到结果链表的最后，pb指向下一个结点 */
```

```
if (pa->expn==pb->expn)
  { x=pa->coef+pb->coef ;
    if (abs(x)<=1.0e-6)
      /* 系数和为0，pa, pb分别直接后继结点 */
      { pa=pa->next ; pb=pb->next ;  }
    else   /* 若系数和不为0，生成的结点插入到结果
链表的最后， pa, pb分别直接后继结点 */
      { p=(ploy *)malloc(sizeof(ploy)) ;
        p->coef=x ; p->expn=pb->expn ;
        p->next=NULL ;
          /* 生成一个新的结果结点并赋值 */
        pc->next=p ; pc=p ;
        pa=pa->next ; pb=pb->next ;
```

```
            }
        }
    }    /* end of while */
if  (pb!=NULL)
    while(pb!=NULL)
      {   p=(ploy *)malloc(sizeof(ploy)) ;
          p->coef=pb->coef ; p->expn=pb->expn ;
          p->next=NULL ;
              /*  生成一个新的结果结点并赋值  */
            pc->next=p ; pc=p ; pb=pb->next ;
      }
```

```
    if  (pa!=NULL)
      while(pa!=NULL)
        {   p=(ploy *)malloc(sizeof(ploy)) ;
            p->coef=pb->coef ; p->expn=pa->expn ;
            p->next=NULL ;
                /*  生成一个新的结果结点并赋值  */
              pc->next=p ; pc=p ; pa=pa->next ;
        }
    return (Lc) ;
}
```

本章小结

熟练掌握：

(1)线性表的逻辑结构特性和线性表的(ADT)设计；

(2)线性表的顺序存储结构和链式存储结构两种实现方式和基本操作；

(3)双向链表的插入和删除等基本操作及相关算法;

(4)循环链表的特点及相关操作;

(5)理解一元多项式的表示方法及相加算法，提高自己用线性表解决实际问题的应用水平。

顺序表

- 优点
  - 存储密度大:无须为表示线性表中元素之间的逻辑关系而增加额外的存储空间。
  - 具有随机存取特性。
- 缺点
  - 插入和删除操作需要移动大量元素。
  - 初始空间大小分配难以掌握。

链表

◆优点

- 由于采用节点的动态分配方式，具有良好的适应性。
- 插入和删除操作只需修改相关指针域，不需要移动元素。

◆缺点

- 存储密度小: 为表示线性表中元素之间的逻辑关系而需要增加额外的存储空间(指针域)。
- 不具有随机存取特性。

思考题

求解约瑟夫问题

- 约瑟夫问题：

  n个人围成一个圆圈，首先，第1个人从1开始，顺时针报数，报到第m个人，令其出列。然后再从下一个人开始，从1顺时针报数，报到第m个人，再令其出列，…，如此下去，直到圆圈中只剩一个人为止。此人即为优胜者。

- 算法基本思路？
- 用什么数据结构？